

KubExporter
(kubernetes) Backup better. Together.

Raphael Müller / Concordia Marc Brugger / Bison Group

Wer sind wir?

- **Marc Brugger aka bakito**: "Das geht besser"-Mindset - und Morgen gibt es die Lösung auf Github
- **Raphael Müller aka Raphi Milla**: "KISS-Prinzip"-Mindset - Einfache Dinge zuerst

Wo alles begann (once upon a time in zürich hardturm quartier)

- Marc
  - begann seinen Kubernetes Entwickler-Journey mit OpenShift 3 und bemängelte das Tooling ...
  - Dann kamen Custom-Ressourcen und Operator welche mit go am besten laufen ...
  - Plötzlich wurde aus OpenShift eine komplett customizde Enterprise Kubernetes Plattform, genau auf die Firma abgestimmt
- Raphi
  - kam aus der Linux-Welt hinzu und fing Feuer am definierten Zustand der Plattform ...
  - Dann kam die Erkenntnis, dass Operator und Reconsilation Loops einem das Leben erleichtern ...
  - Plötzlich konnten Abläufe die in der alten Welt Monate brauchten, innerhalb 24 h abgearbeitet werden

Warum sind wir hier?

- **OpenSource hilft bei der Zusammenarbeit, auch in neuen Firmen** (Marc Bison, Raphi CONCORDIA - auch über Firmengrenzen hinaus/hinweg)
- **OpenSource hilft verbunden zu bleiben** (und wenn nur schon über PR's und Issues)
- **OpenSource hilft sich und das Produkt weiter zu entwickeln** (PR's und MR's für neue Features und Ideen)
- **Opensource hilft anderen Kollegen und Firmen dieselben Probleme anzugehen** (Wie macht Ihr ... ?)
- **Murphy lauert immer und überall - Begegnen wir ihm vorbereitet**
  - **Namespace Configs** (Aber ich dachte, wir hätten ...)
  - **Cluster Upgrades** (War vor dem Upgrade ganz sicher anders)
  - **Fat Fingers** (Ooops I did it again)

Die Lösung: KubExporter

Auf GitHub finden wir genau das Werkzeug, welches uns diese Arbeit abnimmt: 👉 https://github.com/bakito/kubexporter

> "KubExporter allows you to export resources from kubernetes as yaml/json files."

Kubernetes-Cluster sichern: Die Realität

Herkömmliche Backup-Tools eignen sich **nicht** für Container.

- **Schnelllebigkeit**: Container sind transient und ephemeral.
- **Falscher Fokus**: Klassische Datei- oder VM-Backups greifen hier ins Leere.

Gängige Kubernetes Backup-Tools

Spricht man über Kubernetes-Backups, fallen meistens diese Namen:

- **Enterprise- & All-in-One-Lösungen**
  - **Velero** (Der Open-Source-Standard von VMware)
  - **Kasten K10** (Veeam – stark im Enterprise-Umfeld)
  - **TrilioVault** (Fokus auf Cloud-Native & App-Sicherung)
- **Storage- & Snapshot-Fokussiert**
  - **Longhorn** (Eingebautes Backup für verteilten Storage)
  - **OADP** (Red Hat Produkt, welches auf Velero aufsetzt)
  - **Portworx PX-Backup** (Fokus auf Multi-Cloud-Storage)

Persistente Daten vs. Konfiguration

Die genannten Tools sind mächtig, haben aber oft einen Haken: Sie sind darauf ausgelegt, persistente Volumes (PVs) zu sichern.

Oder so bieten nur eine Komplettlösung an. (Backup/Restore alles oder nichts)

Unsere Philosophie:

- Wir wollen möglichst **keine** persistenten Daten innerhalb von Kubernetes betreiben
- **Was wir wirklich brauchen:** Eine reine Sicherung der Config-Ressourcen!

Was macht KubExporter so stark?

- **Automatische Erkennung**: Exportiert standardmässig alle Ressourcen. Neue CRDs werden automatisch erfasst.
- **Flexibilität**: Exportiert wahlweise als **YAML** oder **JSON**.
- **Bereinigung**: Unerwünschte Ressourcen und Standard-Metadaten lassen sich gezielt ausschliessen.

Installation & Verwendung

Schnelle Installation

```
# Als kubectl Plugin via Krew
kubectl krew install exporter

# Oder via Homebrew
brew install --cask kubexporter
```

Einfacher Aufruf

```
# Exportiert standardmässig in den Ordner "exports"
kubectl exporter
```

Kubernetes-Cluster sichern: So einfach gehts

Kubexporter im Schnelldurchlauf

- **Kubexporter Schedulen**: Ein k8s-(Cron)-Job oder ein cli-command und schon wird gebackupt
- **Storage**: Von einem einfachen PV zu S3 und HyperScaler Cloudstorage: alles geht
- **Verschlüsselung**: Secrets aus-sterndlen ***, AES oder AGE verschlüsseln
- **Restore**: Ganze Directories, einzelne Dateien oder einfach nur "güxle" in Git

Warum das alles wirklich praktisch ist

Wieviele Male haben wir schon Configs überschrieben oder nicht mehr im Original gefunden ?

- **Ressourcen Ressourcen Ressourcen**
  - **CR's welche nicht unter Git/Argo Kontrolle waren** (Der Klassiker)
  - **ConfigMaps, Secrets oder andere Ressource-Definitionen welche abhanden kamen** (Soll es auch im professionellen Umfeld geben)
  - **Cluster eigene Ressourcen, die sich geändert haben** (Wer kennt OLM ?)
  - **RBAC, Service-Accounts und Konsorten** (Haben wir das nicht ge-GitOpst ?)
- **Murphy is in the house**
  - **Namespace Configs** (Aber ich dachte, wir hätten ...)
  - **Cluster Upgrades** (War vor dem Upgrade ganze sicher anders)
  - **Fat Fingers** (Ooops I did it again)

Diff is your friend

```
diff -u
snap1/kubexporter-test/ConfigMap.diff-hello.yaml
snap2/kubexporter-test/ConfigMap.diff-hello.yaml
```

```
--- snap1/kubexporter-test/ConfigMap.diff-hello.yaml
2026-10-22 16:19:55.912027563 +0200
+++ snap2/kubexporter-test/ConfigMap.diff-hello.yaml
2026-10-22 16:19:56.125027564 +0200
@@ -1,6 +1,6 @@
 apiVersion: v1
 data:
-  hello: World
+  hello: Lucerne
 kind: ConfigMap
 metadata:
   name: diff-hello
```

Was macht KubExporter so stark?

- **Automatische Erkennung**: Exportiert standardmässig alle Ressourcen. Neue CRDs werden automatisch erfasst.
- **Flexibilität**: Exportiert wahlweise als **YAML** oder **JSON**.
- **Bereinigung**: Unerwünschte Ressourcen und Standard-Metadaten lassen sich gezielt ausschliessen.

Installation & Verwendung

Schnelle Installation

```
# Als kubectl Plugin via Krew
kubectl krew install exporter

# Oder via Homebrew
brew install --cask kubexporter
```

Einfacher Aufruf

```
# Exportiert standardmässig in den Ordner "exports"
kubectl exporter
```

Fazit

- Container-Backups neu gedacht
- Fokus auf das, was zählt: **Die Infrastruktur als Code**
- Schlank, effizient und perfekt für die CI/CD-Pipeline

Ursprung – wo kommt KubExporter her?

- Community-Produkt auf Ruby-Basis wurde verwendet, um Kubernetes-Ressourcen zu sichern
  - Alle Ressourcen mussten explizit in der Konfiguration definiert werden – sonst wurden sie nicht gesichert
  - Ein Export eines Clusters dauerte über eine Stunde
  - Nach Einführung einer neuen CRD im Cluster wurde diese nicht in die Konfiguration aufgenommen → wir hatten keine Backups, wenn wir sie brauchten
- Entscheidung, ein neues Backup-Tool in go zu implementieren
  - Opt-Out statt Opt-In (Alle Ressourcen sind standardmässig enthalten)
  - Exportdauer auf < 30s reduziert
  - Läuft als CronJob → Keine permanenten Ressourcen erforderlich
  - Export nach Git → Vollständige Historie verfügbar

Warum OpenSource

- Grossteil der Zeit wurde privat ausserhalb des Unternehmens investiert
- OpenSource → Community-gesteuerte Entwicklung
- Neue Features werden von der Community angefordert/hinzugefügt
- Für alle zugänglich
  - Apache 2.0-Lizenz
- Integration in bestehende Paket-/Plugin-Manager
  - GitHub
  - 🍺 brew
  - 🔌 krew (kubectl-Plugin)
  - ❄️ nix

Funktionen

- 🚀 Parallele Export-Worker
- 📊 Verschiedene Fortschrittsformate
- ☁️ S3 / Google Storage Bucket-Unterstützung
- 🔐 Secret-Maskierung oder -Verschlüsselung (age/AES)
- 📈 OTLP-Metriken
- 🔄 Owner-Reference-Migration (Wiederimport mit aktualisierten Owner-References)
- Möglichkeit für Integration in git

Wie / wann neue Funktionen hinzufügen?

- ☁️ S3 / Google Storage Bucket-Unterstützung
  - Migration in die Cloud, wo Git keine Option war
- 🔐 Secret-Maskierung oder -Verschlüsselung
  - Bei der Arbeit mit Git ist es nicht sicher, den Inhalt von Secrets zu committen
- 📈 OTLP-Metriken
  - Regression nach Implementierung der Multi-Namespace-Auswahl. KubExporter wurde ohne Fehler beendet, hat aber nichts exportiert → Mit Metriken können wir Alerts basierend auf den Metriken einrichten.
- 🔄 Owner-Reference-Migration
  - Ressourcen, die in einem Cluster importiert wurden – indem die übergeordneten Ressourcen neu erstellt wurden (neue UUID) – wurden aufgrund ungültiger Owner-Reference gelöscht

Demo

(Drücke ctrl + e zum Ausführen)

```
rm -rf ${EXPORTS}
cd "${PRESENTATION_ROOT}/kubexporter" && clear
kubectl exporter --config demo-config.yaml -p bar --summary=false
echo
read -n 1 -s -r -p "Press any key to return to the presentation ... "
```

snippet +exec is disabled, run with -x to enable

Verschlüssen von Secrets mit age - Secret unverschlüsselt

```
cd "${PRESENTATION_ROOT}/kubexporter"
cut -c1-80 ${SECRET} | sed -n '1,4p' | bat -l yaml
```

snippet +exec is disabled, run with -x to enable

Verschlüssen von Secrets mit age

Bestehende Dateien nachträglich verschlüsseln – nur der **öffentliche** Schlüssel des Empfängers wird benötigt:

```
cd "${PRESENTATION_ROOT}/kubexporter"
rm -f age.key && age-keygen -o age.key 2>/dev/null
PUB=$(age-keygen -y age.key)
kubectl exporter encrypt --age-public-key "${PUB}" ${SECRET}
```

snippet +exec is disabled, run with -x to enable

Verschlüssen von Secrets mit age - Secret verschlüsselt

```
cut -c1-80 ${SECRET} | sed -n '1,14p' | bat -l yaml
```

```
snippet +exec is disabled, run with -x to enable
```

Entschlüsselte von Secrets mit age

```
kubectl exporter decrypt \
  --age-identity "${PRESENTATION_ROOT}/kubexporter/age.key" \
  ${SECRET}
```

snippet +exec is disabled, run with -x to enable

Verschlüssen von Secrets mit age - Secret unverschlüsselt

```
cut -c1-80 ${SECRET} | sed -n '1,4p' | bat -l yaml
```

```
snippet +exec is disabled, run with -x to enable
```

Use case: diff two snapshots

```
cd /tmp/kubexporter-demo
rm snap* -Rf
kubectl -n ${NAMESPACE} delete configmap diff-hello >/dev/null
kubectl -n ${NAMESPACE} delete configmap diff-hoi >/dev/null

kubectl create configmap diff-hello --from-literal=hello=World \
  -n ${NAMESPACE}

echo
kubectl exporter -n ${NAMESPACE} -i ConfigMap -t snap1 --clear-target --quiet

kubectl create configmap diff-hoi --from-literal=hoi=Lozärn \
  -n ${NAMESPACE}
kubectl patch configmap diff-hello -p '{"data":{"hello":"Lucerne"}}' -n ${NAMESPACE}

echo
kubectl exporter -n ${NAMESPACE} -i ConfigMap -t snap2  --clear-target --quiet
```

snippet +exec is disabled, run with -x to enable

Use case: diff two snapshots

```
cd /tmp/kubexporter-demo
kubectl exporter -n ${NAMESPACE} -i ConfigMap -t snap2 -c -q
diff -ru --color=always snap1 snap2
```

snippet +exec is disabled, run with -x to enable

Demo OTel (OpenTelemetry) Metrics

If configures, metrics will be pushed to the OTel collector if the sync is finished.

Metric	Description
kubexporter.duration_seconds	Total export duration in seconds
kubexporter.errors	Number of errors encountered during export
kubexporter.exported_resources	Total number of exported resources
kubexporter.exported_size_bytes	Total size of exported resources in bytes
kubexporter.kinds	Number of kinds processed
kubexporter.namespaces	Number of namespaces containing exported resources
kubexporter.query_pages	Total number of query pages requested
kubexporter.resource.export_duration_seconds	Export duration per kind in seconds
kubexporter.resource.exported_instances	Number of exported resource instances per kind
kubexporter.resource.exported_size_bytes	Size of exported resources per kind in bytes
kubexporter.resource.instances	Number of resource instances found per kind
kubexporter.resource.query_duration_seconds	Query duration per kind in seconds
kubexporter.resource.query_pages	Number of query pages per kind

```
kubectl logs -l app=otel-collector -n otel --tail=-1 | grep exported.instances | cut -c40-170
```

snippet +exec is disabled, run with -x to enable

KubExporter - (kubernetes) Backup better. Together.

- Firmen-internes Problem
- Initiative durch Mitarbeiter etwas Neues zu entwickeln
- Publikation als OpenSource Projekt bringt Nutzen auch für andere.
- Zusammen können wir das Tool weiter verbessern

Warum KubExporter?

- Backup / Disaster Recovery – ein Snapshot aller Ressourcen eines Clusters als einfache Dateien
- Audit & Diff – Cluster untereinander oder denselben Cluster im Zeitverlauf vergleichen
- Migration – Workloads zwischen Clustern / Providern verschieben
- Debugging – den vollständigen Zustand eines Namespace für die Offline-Analyse sichern
- Manifest-Extraktion – manuell erstellten Cluster-Zustand in wiederverwendbare Manifeste umwandeln
- Ausführbar ad-hoc – in einer Pipeline oder als CronJob

Die Ausgabe besteht nur aus Dateien: Sie lassen sich auf der Festplatte, in einem tar.gz, in S3 / GCS oder als Files in Git ablegen.

Fragen? - Probier mich aus!

KubExporter: https://github.com/bakito/kubexporter

Slides: https://killercoda.com/bakito/scenario/devcon-lozaern

```
echo "
https://killercoda.com/bakito/scenario/devcon-lozaer
n" | qrencode -t utf8i
```

snippet +exec_replace is disabled, run with -X to enable

Informationen zur Umgebung

```
echo -e "NAMESPACE:"
echo -e "  ${NAMESPACE}\n"
echo -e "PRESENTATION_ROOT:"
echo -e "  ${PRESENTATION_ROOT}\n"
echo -e "EXPORTS:"
echo -e "  ${EXPORTS}\n"
echo -e "SECRET:"
echo -e "  ${SECRET}"
```

snippet +exec_replace is disabled, run with -X to enable